

Vehicle Centerline Calibration

Vehicle Centerline Calibration

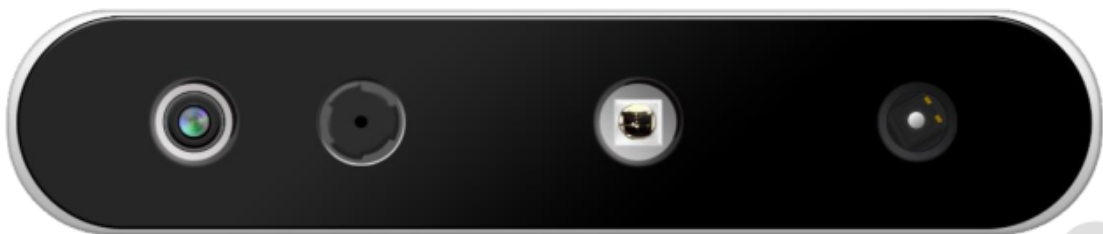
1. Course Content
2. Run Example Program
3. Calibrate Vehicle Body Centerline
4. Calibrate Lane Reference Lines
 - 4.1 Calibrate Left Lane Reference Line
 - 4.2 Calibrate Right Lane Reference Line
5. Source Code Analysis
 - 5.1 Calibration Parameter File
 - 5.2 Calibration Program

1. Course Content

1. Start the example calibration program to calibrate the vehicle body centerline and the reference lines for left and right lane lines in the camera perspective.

[!NOTE]

- This calibration tutorial is only for users who are using the autonomous driving function for the first time to calibrate according to their camera model.
 - The visual autonomous driving function uses the camera's RGB image and does not rely on depth information.
 - ROSMASTER-M3 vehicle can be optionally equipped with two depth cameras. Due to the different positions of the **RGB cameras** of the two depth cameras, using different cameras will result in different positions of the vehicle body centerline in the camera perspective, so it is necessary to calibrate the vehicle body's centerline in different camera perspectives.
-
- Angstrong Nuwa-HP60C



- Orbbec DaBai DCW2



2. Run Example Program

- Start camera node

```
ros2 launch m3_bringup bringup_camera.launch.py
```

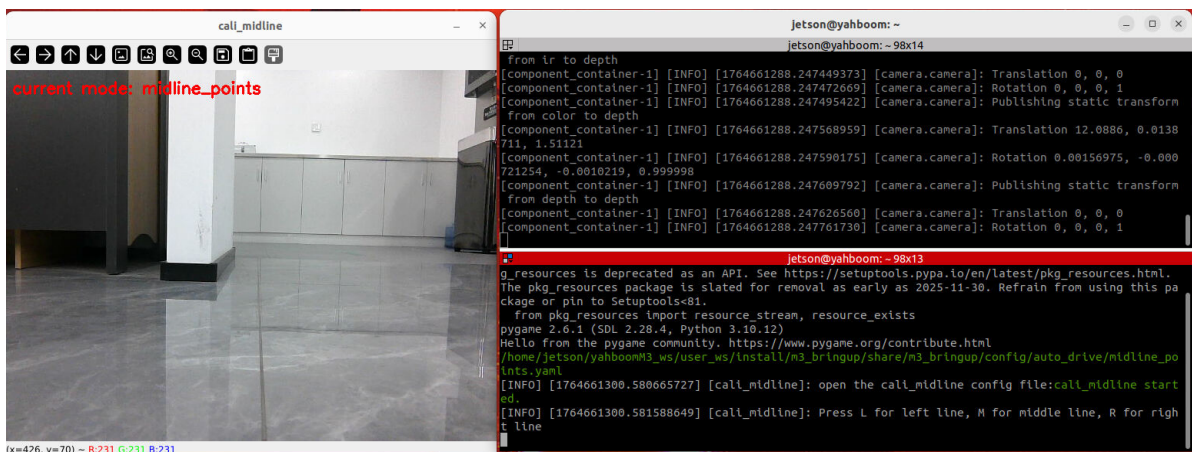
[!IMPORTANT]

If you have not set the camera's `CAMERA_TYPE` environment variable, please first set the camera environment variable according to this chapter's tutorial `01-Site Layout` and `Precautions`

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
[System Information]
ROS: humble
DOMAIN_ID: 15
IP_Address_1: 192.168.10.169
IP_Address_2: 172.19.0.1
LASER_TYPE: single
CAMERA_TYPE: as_hp60c
jetson@yahboom:~$ ros2 launch m3_bringup bringup_camera.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-09-31-50-009509-yahboom-81615
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ascamera_node-1]: process started with pid [81669]
```

- Start centerline calibration program

```
ros2 run auto_drive cali_midline
```

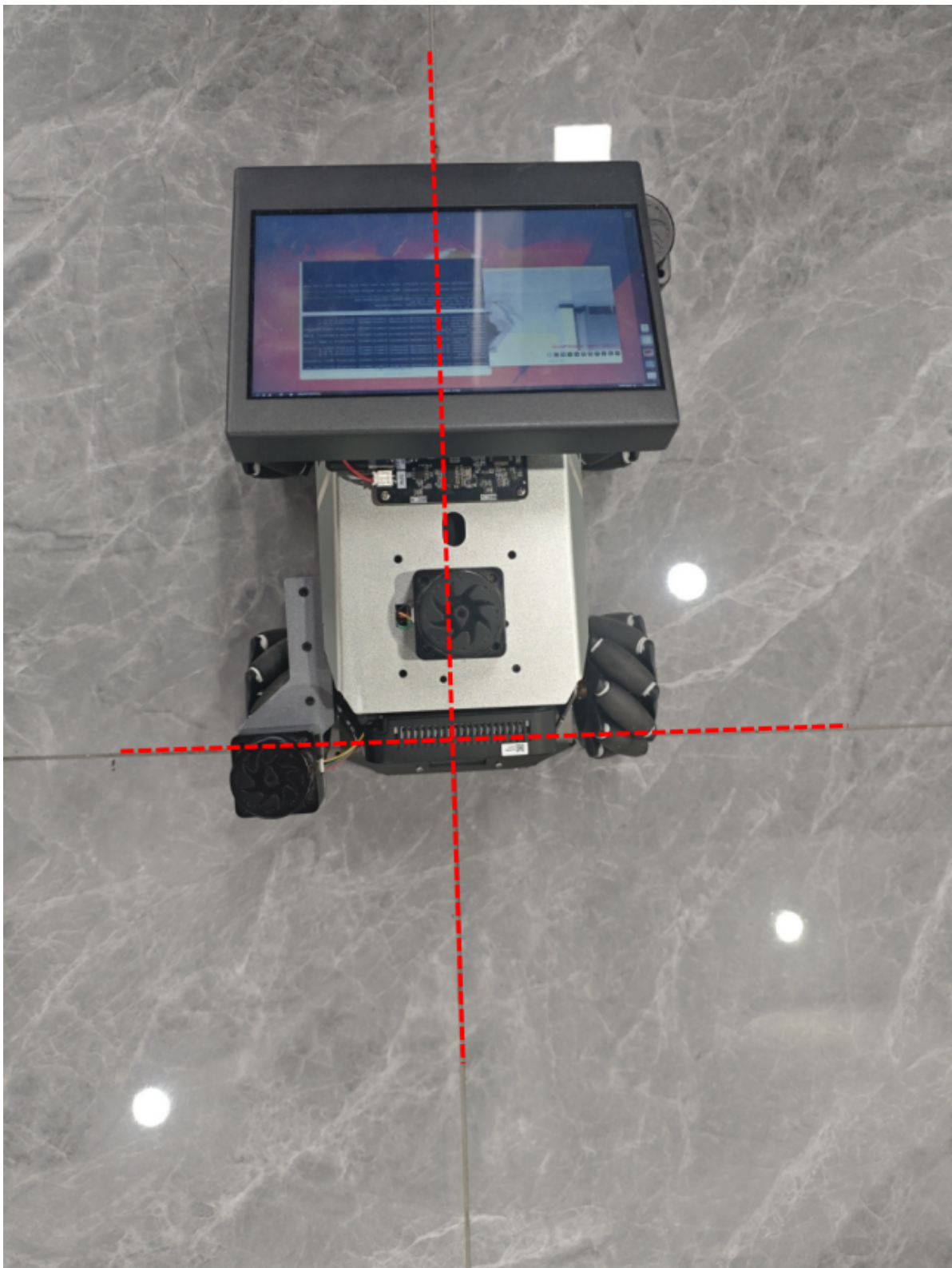


3. Calibrate Vehicle Body Centerline

- Adjust the camera to normal use position (camera facing forward, avoid camera angle being too tilted up or down)



- Then find any reference line (any relatively straight line will work, here we use the tile grout line as reference), align the vehicle body center with the reference line



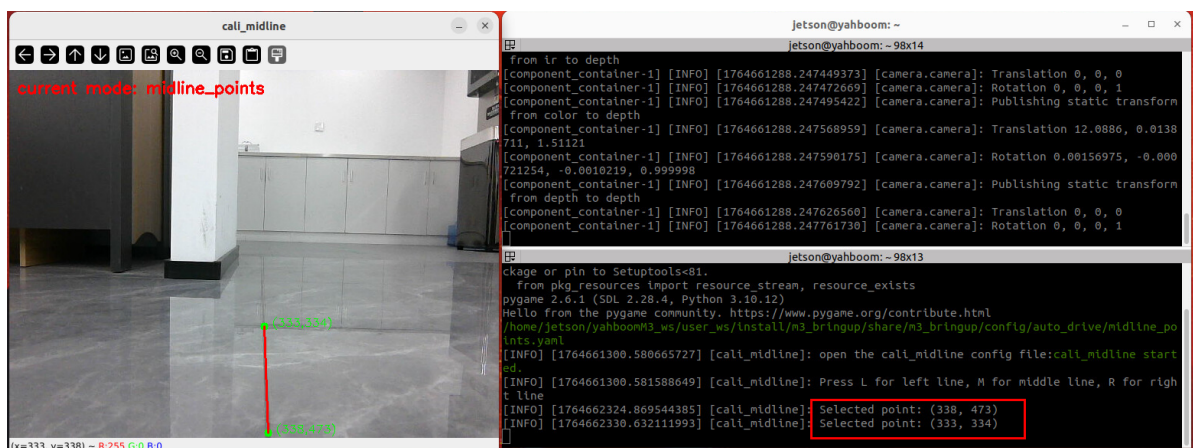
- In the camera image, we can see the reference line (tile grout line), which is the position of the vehicle body's centerline in the current camera perspective. The `current mode` in the upper left corner shows the current calibration mode, `midline_points` indicates calibrating the vehicle body centerline.

[!NOTE]

Different cameras will show differences in the vehicle body centerline due to different camera positions. It can be found that the vehicle body centerline is not completely vertical in the camera perspective, but is actually a diagonal line.



- Use the mouse to select the start and end points of the centerline in sequence. Select any two points from bottom to top on the reference line. The length does not need to be considered, as it can be adjusted through parameters later. After selection is completed, the terminal will print the coordinates of the mouse-selected start and end points.



- Then press the S key on the keyboard to save. The terminal will prompt the file location where the parameters are saved.

[!CAUTION]

- The file save path prompted by the terminal

~/yahboomM3_ws/user_ws/install/m3_bringup/share/m3_bringup/config/auto_dri

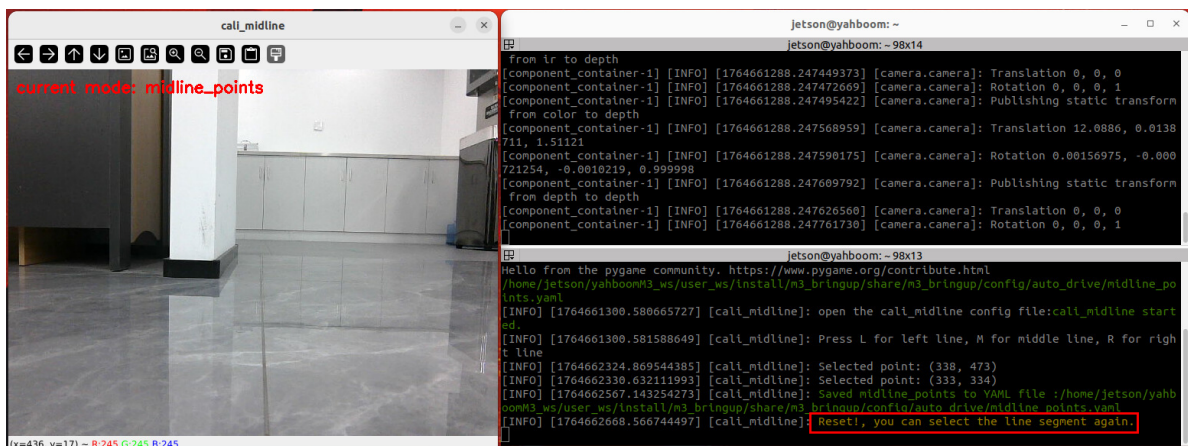
ve/midline_points.yaml is the install installation directory of the m3_bringup package.

- However, the m3_bringup package uses colcon build --symlink-install for symbolic link compilation, so the actual save path of the parameters is the source code path of the m3_bringup package:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/auto_drive/midline_points.yaml
```

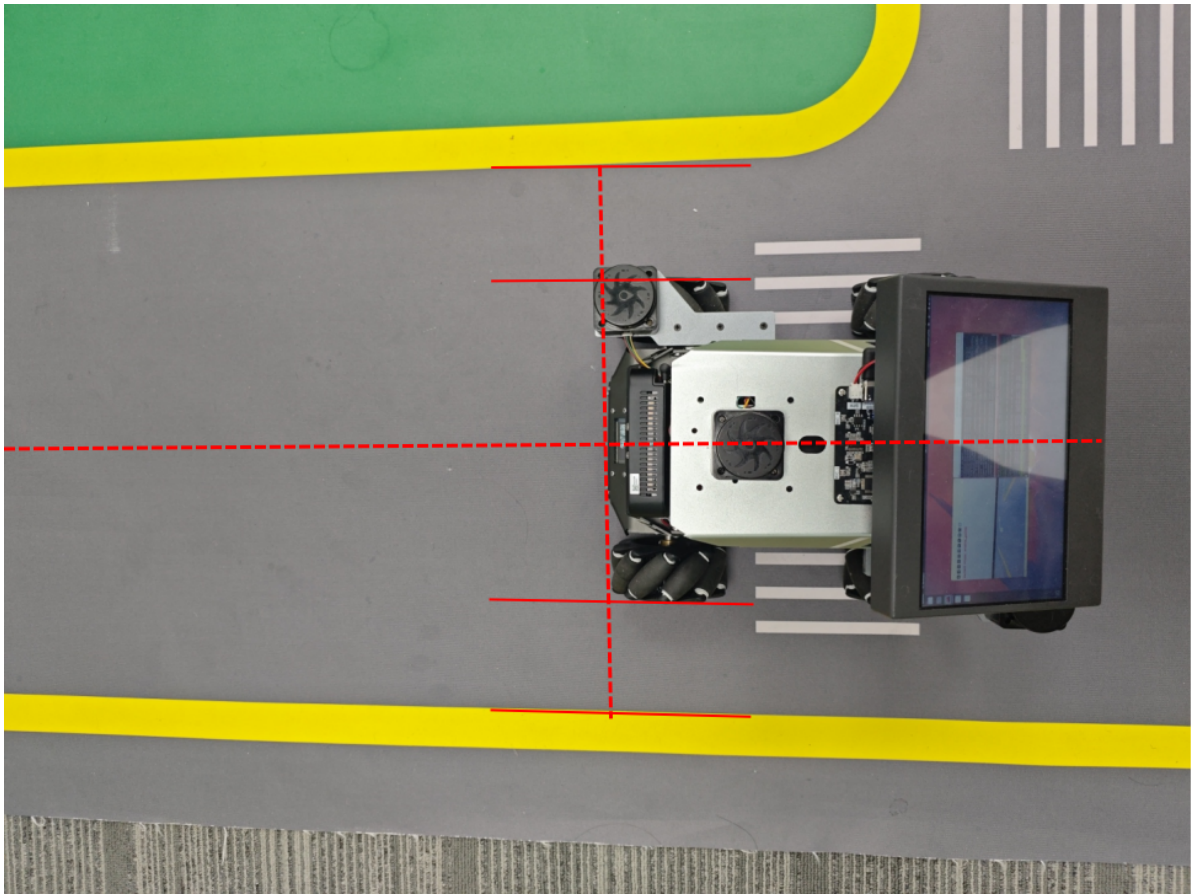
```
jetson@yahboom: ~ 98x13
pygame 2.6.1 (SDL 2.28.4, Python 3.10.12)
Hello from the pygame community. https://www.pygame.org/contribute.html
/home/jetson/yahboomM3_ws/user_ws/install/m3_bringup/share/m3_bringup/config/auto_drive/midline_points.yaml
[INFO] [1764661300.580665727] [cali_midline]: open the cali_midline config file:cali_midline started.
[INFO] [1764661300.581588649] [cali_midline]: Press L for left line, M for middle line, R for right line
[INFO] [1764662324.869544385] [cali_midline]: Selected point: (338, 473)
[INFO] [1764662330.632111993] [cali_midline]: Selected point: (333, 334)
[INFO] [1764662567.143254273] [cali_midline]: Saved midline_points to YAML file :/home/jetson/yahboomM3_ws/user_ws/install/m3_bringup/share/m3_bringup/config/auto_drive/midline_points.yaml
```

- If you are not satisfied with the calibration result or clicked by mistake, you can press the R key on the keyboard to reset the calibration state and recalibrate.



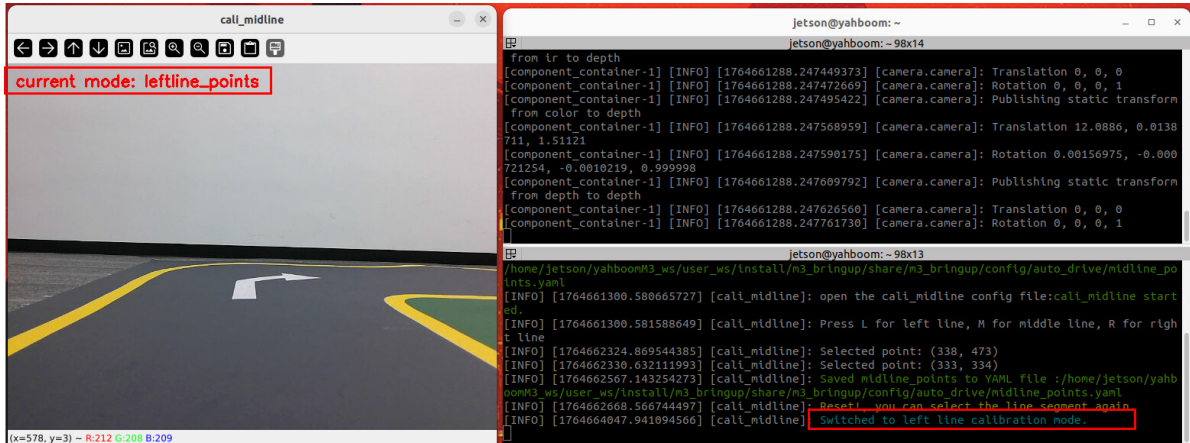
4. Calibrate Lane Reference Lines

- Place the vehicle in the lane of the sandbox map
 - The vehicle body centerline should be as close as possible to the centerline position of the lane
 - The distance between the vehicle body and the left and right lane lines should be as equal as possible.

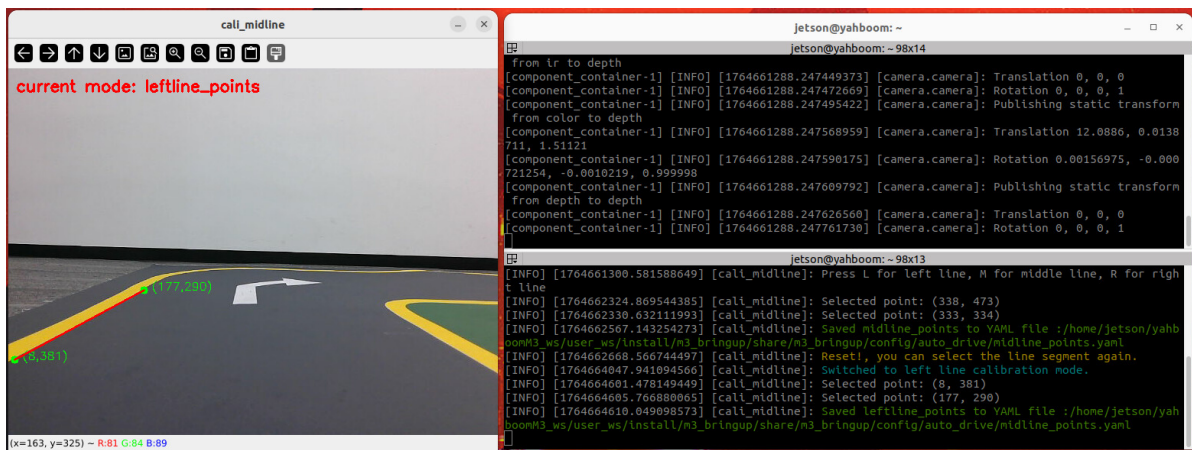


4.1 Calibrate Left Lane Reference Line

- Click on the title of `cal_midline`, then press the L key on the keyboard to switch to `leftline_points` calibration mode

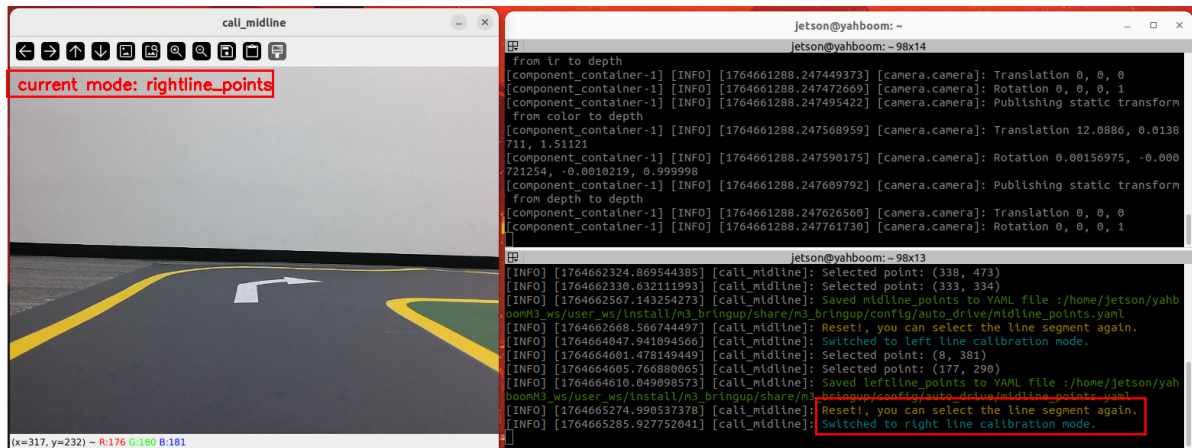


- The calibration method is similar to before. Select the start and end points on the inner side of the left lane line in sequence, then click the S key to save. The terminal will print prompt information.

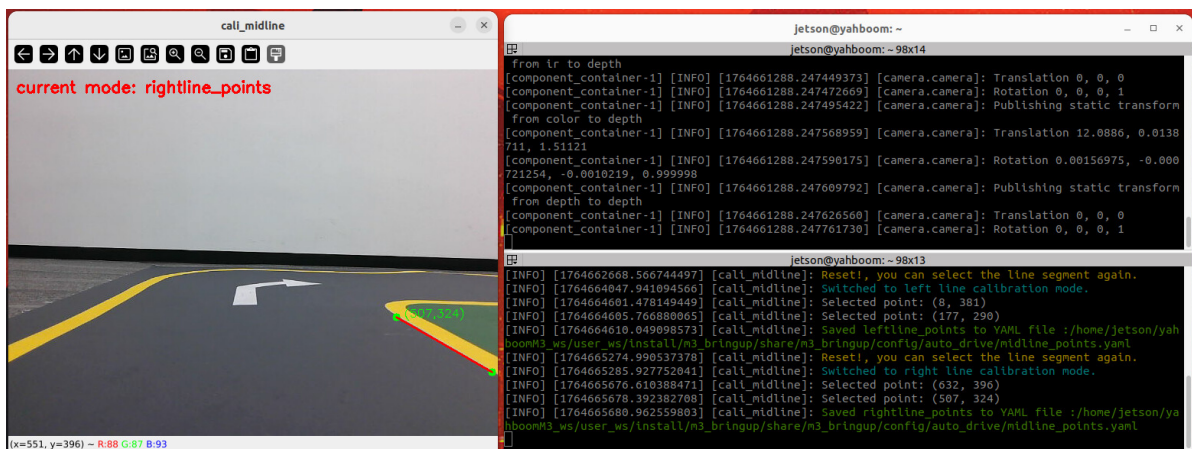


4.2 Calibrate Right Lane Reference Line

- First press the R key on the keyboard to reset the calibration state, then press the K key to enter `rightline_points` calibration mode



- The calibration method is similar to before. Select the start and end points on the inner side of the right lane line in sequence, then click the S key to save. The terminal will print prompt information



[TIP]

- It can be found that when the vehicle is in the middle of the lane, the left and right lane lines in the camera perspective are actually two line segments extending toward the

inside of the lane, and in different camera perspectives, the inclination degree of the lane lines will also be different.

5. Source Code Analysis

5.1 Calibration Parameter File

- All calibrated parameters are saved in the `midline_points.yaml` parameter file of the `m3_bringup` package. The file path is:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/auto_drive/midline_points.yaml
```

- The `midline_points` field is the data of the vehicle body centerline
- The `leftline_points` field is the left lane line reference line in the camera perspective
- The `rightline_points` field is the right lane line reference line in the camera perspective
- The `midline_length` field is the length of the vehicle body centerline (generally use the default value, parameter adjustment will be explained in later tutorials)

```
midline_points:
  start_point:
    x: 338
    y: 473
  end_point:
    x: 333
    y: 334
leftline_points:
  start_point:
    x: 8
    y: 381
  end_point:
    x: 177
    y: 290
rightline_points:
  start_point:
    x: 632
    y: 396
  end_point:
    x: 507
    y: 324
midline_length: 115
```

5.2 Calibration Program

- The calibration program is in the `cali_midline.py` program file of the `auto_drive` package. The file path is:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/cali_midline.py
```

- The core program is in the `Cali_Midline` class
1. Receive RGB camera images published by ROS topics;
 2. Three calibration modes: "left line / middle line / right line" (default is middle line);
 3. Mouse interaction calibration: select 2 endpoints through left mouse button clicks, automatically draw line segments;
 4. Real-time display of selected points, line segments, and current calibration mode;
 5. Data persistence: save calibrated line segment endpoints to YAML file, while preserving other existing configurations in the file;
 6. Support keyboard shortcuts for reset selection, switch modes, and save data.
- Node initialization: `init`
 - Receive images, format conversion, visualization drawing, keyboard interaction: `image_callback`
 - Capture left mouse button clicks, collect line segment endpoints: `mouse_callback`
 - Validate and save calibration data to YAML file: `save_to_yaml`

```
class Cali_Midline(Node):
    def __init__(self):
        super().__init__('cali_midline')
        rgb_topic=get_rgb_topic()
        self.declare_parameter("config_file",
os.path.join(get_package_share_directory("m3_bringup"),
'config','auto_drive',"midline_points.yaml"))
        self.config_file =
self.get_parameter("config_file").get_parameter_value().string_value
        print(Fore.GREEN+self.config_file+Fore.RESET)

        # ros communication
        self.subscription = self.create_subscription(Image, rgb_topic,
self.image_callback, 5)
        self.bridge = CvBridge()

        self.points = []
        self.line_drawn = False
        self.line_type = "midline_points"

        cv2.namedwindow("cali_midline")
        cv2.setMouseCallback("cali_midline", self.mouse_callback)
        self.get_logger().info('open the cali_midline config
file:'+Fore.GREEN+"cali_midline started."+Fore.RESET)
        self.get_logger().info('Press L for left line, M for middle line, R for
right line')

    def image_callback(self, msg):
        # Convert ROS images to OpenCV format
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        for (x, y) in self.points:
            cv2.circle(cv_image, (x, y), 5, (0, 255, 0), -1) # Green dots
            cv2.putText(cv_image, f"({x},{y})", (x+10, y),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

        if self.line_drawn and len(self.points) == 2:
            cv2.line(cv_image, self.points[0], self.points[1], (0, 0, 255), 2)
```



```

cv2.putText(cv_image, f"current mode: {self.line_type}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

cv2.imshow("cali_midline", cv_image)
key = cv2.waitKey(1) & 0xFF

if key == ord('r'):
    self.points = []
    self.line_drawn = False
    self.get_logger().info(Fore.YELLOW+"Reset!, you can select the line
segment again."+Fore.RESET)
elif key == ord('s'):
    self.save_to_yaml(self.config_file)
elif key == ord('l') or key == ord('L'):
    self.line_type = "leftline_points"
    self.get_logger().info(Fore.CYAN+"Switched to left line calibration
mode."+Fore.RESET)
elif key == ord('m') or key == ord('M'):
    self.line_type = "midline_points"
    self.get_logger().info(Fore.CYAN+"Switched to middle line
calibration mode."+Fore.RESET)
elif key == ord('k') or key == ord('K'):
    self.line_type = "rightline_points"
    self.get_logger().info(Fore.CYAN+"Switched to right line calibration
mode."+Fore.RESET)

def mouse_callback(self, event, x, y, flags, param):
    """Mouse callback function to handle click events"""
    if event == cv2.EVENT_LBUTTONDOWN:
        if len(self.points) < 2:
            self.points.append((x, y))
            self.get_logger().info(f"Selected point: ({x}, {y})")
            # Draw a straight line after selecting both points.
            if len(self.points) == 2:
                self.line_drawn = True

def save_to_yaml(self, file_path):
    """Save the endpoints of the line segment to a YAML file."""
    if len(self.points) != 2:
        self.get_logger().info(Fore.RED+"Please select two points to define
the line segment!" +Fore.RESET)
        return
    if os.path.exists(file_path):
        with open(file_path, 'r') as f:
            data = yaml.safe_load(f) or {}
    else:
        data = {}

    if self.line_type not in data:
        data[self.line_type] = {}
    data[self.line_type]["start_point"] = {"x": self.points[0][0], "y":
self.points[0][1]}
    data[self.line_type]["end_point"] = {"x": self.points[1][0], "y":
self.points[1][1]}

```

```
        with open(file_path, 'w') as f:
            yaml.dump(data, f, sort_keys=False, default_flow_style=False,
allow_unicode=True)
            self.get_logger().info(Fore.GREEN+f"Saved {self.line_type} to YAML file
:{file_path}" + Fore.RESET)
```